

**Portfolio Project: Developing Data Sets for Study of Relationship of GDP to Primary
Education Enrollment**

G. Keith Watson

Colorado State University Global

ISM505 Data Wrangling

Instructor: Dr. Khairi Mohd

Due September 13, 2026

Table of Contents	
Section	Page
Introduction	2
Study Goals	2
Implementation	4
Results	5
Conclusion	7
References	8
Appendix – BarChart ScreenShots	9
Appendix - HTML notebook code and code output	10

Introduction

The purpose of this assignment is both to review and practice data wrangling skills developed in the course of the work due in this Data Wrangling course, and to explore the possible relationship between a country's primary education enrollment levels and its Gross Domestic Product (GDP). Primary education is not specifically vocational, so it would not itself have a direct effect on economic competencies of the population, although it is necessary preparation for acquiring them. GDP provides the public resources for educational infrastructural development, so it could be considered a prerequisite for strong enrollment numbers. A country's GDP depends on many factors, including public policy and law, culture, traditions, and the vocational education and skill sets which the population makes available for business operations, so it would initially appear that they are mutually dependent measures.

In a previous data wrangling project, it was discovered that the primary driver of primary education enrollment in another developing country, India, was new law enacted to both require and guarantee primary education for all children (Parliament of India, 2009). Data from the World Bank Group and UNESCO was also used to come to this conclusion (UNESCO, 2026 & World Bank Group, 2026). This study broadens the inquiry to Argentina and other countries the reader applies the attached Python program to.

Study Goals

The purpose of primary education is to provide foundation skills in reading, writing, and mathematics and thereby to establish a solid foundation for learning. It is typically provided to children between the age of five to twelve years, although its duration varies in different countries. In the United States (US), all years from 1st grade through the 8th, or just before the first year of high school, are considered to be primary education years. In Argentina, primary education generally starts at the age of six and lasts six to seven years, making its duration comparable to that in the US. Argentina is a developing country with considerable political and

economic turmoil in recent decades, so it is a good subject of study for finding the factors impeding its progress. The goal of this project was to produce a clean set of data for a selected country which included its enrollment in primary education next to its Gross Domestic Product for the years 2020 through 2021. With regard to production of meaningful comparisons to study the relationship between these two measures, it is likely valuable to see the same data for various countries and compare these, so the program that was written was designed for this purpose. Viewing time sequenced par charts of each with common scaling was illuminating, as it showed a consistent high performance in education in developing Asian countries with low GDPs compared to others. Primary education enrollment levels appear based on this initial country selection phase therefore to be more dependent on culture and public policy than on GDP.

The World Bank provides the data needed for this purpose in the form of World Development Indicators (WDI), each measuring a different attribute of a country or region relevant to assessment of its economic development and development potential. Some example WDIs are Access to electricity (% of population), Adequacy of social safety net programs (% of total welfare of beneficiary households), Adjusted net national income (current US\$). There are a total of 1442 different measurements collected by the World Bank into the data set provided, which include educational enrollment and completion at various levels. The downloadable CSV formatted data provided fortunately is fairly clean, much of the initial wrangling necessary being done before publication, although checking for possible data wrangling requirements is still needed before final organization of selected contents for particular purposes. This report describes the process and methods used and the results obtained.

Implementation

The primary data management problems in a case where the data is already cleaned and consolidated into a master file in a convenient format, specifically a comma separated values (CSV) text file, were primarily those of dealing with possible missing values, selecting the particular data to include in study data sets, and formatting the data as a time series for tracking of changes in the subject measurements. Loading the the two CSV files into Pandas DataFrames, having studied their contents using a GUI spreadsheet application, made the inspection, cleaning and formatting steps straight forward. Using partial Pandas DataFrame views such as `head()` to do initial examination of the downloaded data, its dimensions and basic characteristics could be determined. This was necessary for instance to check data types of the required fields and to converting those required to appropriate ones for study use. The year values needed to establish the time sequence for the two WDIs required were columns headers in both the World Bank WDI master table and in the primary education by country data table provided. These needed conversion into integer values for proper treatment in any analytics that might be applied. Examining the data provided for completeness respecting study requirements, it was found that the study year range of 2001-2020 was all present, the years available being from 1960 to 2020. Although values for numerous years in the 1960-1999 range were missing, all were present for both the WDI of interest for 2001-2020, with the exception of 2020 for the primary education enrollment number.

Since this the products of this data wrangling and organization project could be used as inputs into various studies, all available years were included in the SQLite database file produced and its parallel CSV file. Also, the same data could be extracted and formatted for any selection of country present in the data files provided, preferably on a single program run so that the 200 megabytes of data files require would only need to be loaded once. Including all of the years present required handling of missing data items in a way that would not without damaging

study results. Since in the loaded DataFrames the missing values were represented as 'nan' values, these could easily be replaced with Null values when producing the SQLite table.

One of the primary preparation problems was selection of data and formatting it as a time sequence. Rather than the columns each representing a year with the measurements for the various WDI listed in that column, years needed to be a column in the output table and the WDI names needed to be used as column headers or field names. This involved visually inspecting the data in a spreadsheet program to select the row that contained the years, then, having selected the row from the DataFrame, filtering out the column values that were not in year columns, leaving one list of years and a parallel list of WDI values from each data file. These could then be traversed for insertion into the DB table created with the three required columns, Year, Primary Enrollment, and GDP. This pivot making row data column data was fairly simple using Python, although it would also be possible using an electronic spreadsheet program like Excel.

Having produced the required tables for a number of countries, bar charts were then constructed and inspected for each country using a class which was generated using Claud.ai. Scaling the education data identically for various countries it was easy to see that primary education enrollment did not depend on GDP since many Asian developing countries with low GDPs had very high primary enrollment numbers. Two example barcharts are included in Appendix – BarChart ScreenShots, one for Argentina and one for Cambodia.

Results

The Search Assist AI feature of the DuckDuckGo search engine was queried regarding the economic and educational progress Argentina has made in recent decades. The following was its response:

Argentina has made notable strides in economic development over the past three decades, particularly through investments in education. The expansion of early childhood education has been a key factor, leading to improved educational outcomes and long-term economic benefits. (DuckDuckGo, 2026)

This particular study using World Bank data leads to entirely different conclusions. The numbers of children enrolled in primary education has remained steady from 2001 through 2020 at just under five million, which according to [OurWorldinData.org](https://ourworldindata.org) is nearly 100% of the population of children aged 5 to 12 years, Argentina's primary education years (Ritchie, 2026). The number of children aged 5 to 14 years has remained fairly constant during the same period at just under 7 million, so assuming an equal number of children in each age group, the number of 5-12 year old children is 8/10 (6.9 million) which is about 5.5 million. This means that Argentina has a healthy 90% enrollment rate at the primary education level which is nearly constant during the study period. The political and economic turmoil of the country is evident in fluctuations of its GDP over the period. Overall considerable growth is evidenced, but the path has been turbulent. For instance, in 2002 the GDP dropped from its level in 2001 of \$268.7B to a mere \$97.7B. However, it recovered steadily over the next five years to its previous level and grew fairly steadily from there to its peak in 2017 of \$643.6B. Since then it has fluctuated widely but none of this turbulence correlates with its maintenance of its primary education level.

Conclusion

An evaluation of how well the objectives of this project were met must include not only whether the required data structures for study were produced but also proof of their utility. The visuals included in Appendix – BarChart ScreenShots provide an easy summary of the trends in the selected WDI answering the study question, whether there is any correlation between GDP of a country and its level of primary education enrollment. There appears to be none for Argentina, and comparison with other counties with very different religious and cultural histories shows there is no correlation more generally.

For future development of the capabilities and generality of the code to support other studies should be done. The code must be reusable for different countries and modification of it to accept arguments specifying the particular WDI to include in a country's study table should be straight forward. These goals were accomplished with the code base developed and submitted. Future studies should include other WDI which may either be influential respecting the two chosen for this study or which would be suspected to be influenced by one or the other or both.

References

Parliament of India (2009), Right of Children to Free and Compulsory Education Act, 2009,
Retrieved on July 22, 2026 from
https://www.indiacode.nic.in/bitstream/123456789/13682/1/rte_act_2009.pdf.

Ritchie, H. (2026) - "Population and Demography", OurWorldinData.org. Retrieved from:
'<https://ourworldindata.org/profile/population-demography/argentina>'.

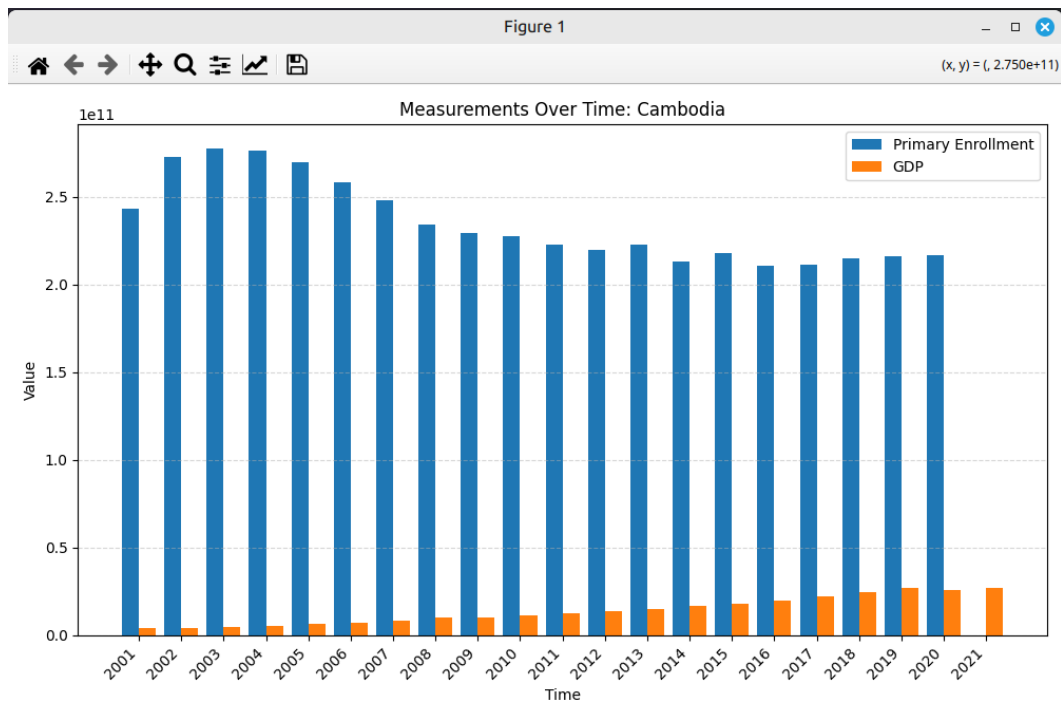
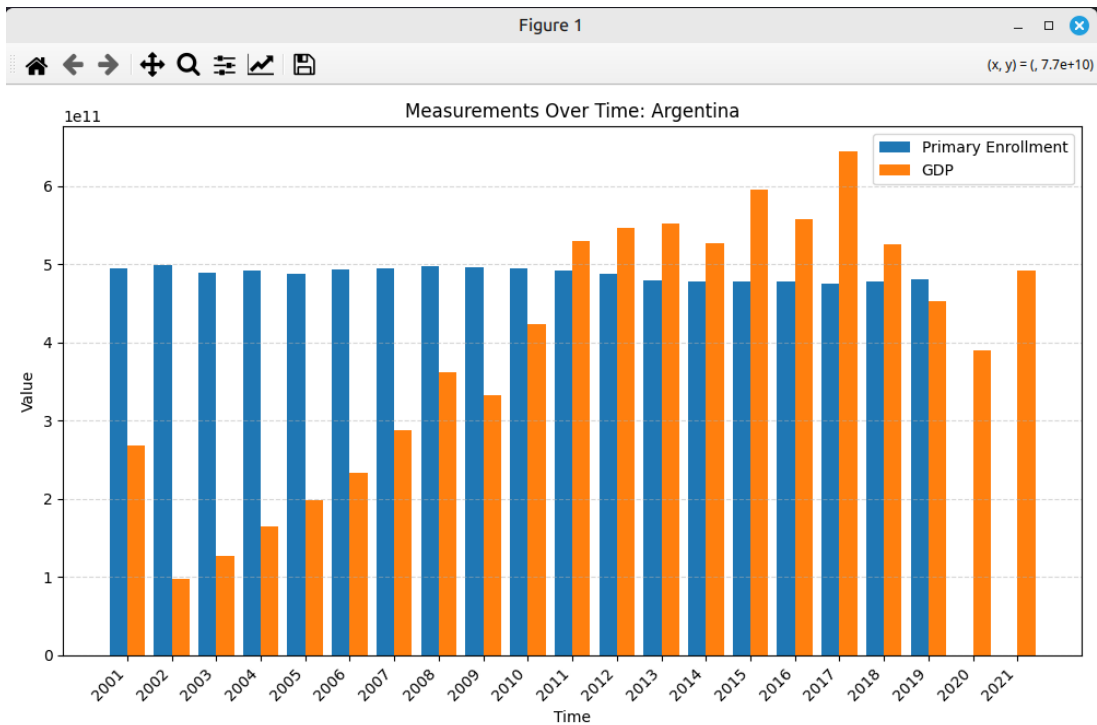
UNESCO (2026), Skills Move Us: How technical training jumpstarts young people's lives,
www.unesco.org, Retrieved on July 22, 2026 from <https://www.unesco.org/en/articles/skills-move-us-how-technical-training-jumpstarts-young-peoples-lives>.

UNESCO Institute for Statistics (UIS) (2025), World Education Statistics 2025,
www.uis.unesco.org, Retrieved on July 22, 2026 from
<https://www.uis.unesco.org/en/publication/world-education-statistics-2025>.

UNESCO Institute for Statistics (UIS) (2026), Background information on statistics in the UIS
database, www.uis.unesco.org, Retrieved on July 23, 2026 from
<https://download.uis.unesco.org/bdds/202602/UIS%20-%20Data%20Release%20Background%20Doc%20-EN-Feb%202026.pdf>

World Bank Group (2026), GDP per capita (current US\$) - India, data.worldbank.org, Retrieved
on July 22, 2026 from <https://data.worldbank.org/indicator/NY.GDP.PCAP.CD?end=2025&locations=IN&start=1960&view=chart>

Appendix – BarChart ScreenShots



Appendix - HTML notebook code and code output

```
# University:    Colorado State University Global
# Course:       MIS 505 - Data Wrangling
# Term:        26FA
# Student:      George Keith Watson
# Student ID:   354019
# Assignment:   Module 8 - Portfolio Project
# Date Due:    September 13, 2026
#
# Workflow Documentation:
#   0. Instantiation of classes is not anticipated in this script, so for security this program
#       is written using @staticmethod's in its classes which have no __init__() method.
#       The code works as a script to be run each time a set of GDP and primary education
#       data is needed for a particular selection of countries.
#   1. When this program is started, the first routine to run is PortfolioProject.load_WDI().
#       This method, along with the next to be run, PortfolioProject.load_PRM_ENRL(),
#       loads the data provided for this assignment as downloaded CSV files. CSU Global provided
#       the data files for this assignment, WDIData.csv, which is the world development indicator
#       (WDI) data from the World Bank, and API_SE.PRM.ENRL_DS2_en_csv_v2_4538564.csv, which
#       is the primary education enrollment data for each country, data also included in the
#       WDIData.csv file under the same WDI code/identifie: SE.PRM.ENRL.
#   2. So that the data files, which are over 200 MB in size, do not need to be loaded for each
#       country the user want to extract the subject WDI from, the DataFrames constructed
#       on load are stored in class level variables in the PorforlioProject class.
#       On particular script run, any number of countries' data can be extracted and stored
#       to the required SQLite DB and CSV files.
#   3. The next method to be run is CountryData.initializeStorage(), which produces an SQLite
#       database file named CountryIndicatorDb in the output folder, "production data",
#       Various useful data extraction and reorganizing tables are built in this DB.
#       This particular method does an integrity check to see if all of the WDI are present
#       for all countries in the database, which the data passes, and then constructs the
#       IndicatorNameMap table, which contains a simple map of the indicator codes to their
#       names.
#   4. The next method to be run is PortfolioProject.makeAssignment_8_DB_n_CSV('Argentina'), which
#       builds the output files required for this assignment, an SQLite database file and a
#       CSV file with the fields "Year", "GDP", and "Primary Enrollment". GDP is stated in
#       current U.S. Dollars, and Primary Enrollment is simply the total number of students
#       enrolled in primary education in that year.
#       The __main__ script of this program can produce any number of country specific data files
#       in a single run, loading the master data files only once, by appending calls to
#       PortfolioProject.makeAssignment_8_DB_n_CSV( CountryName ) to the end of this script.
#
# The next step in development will be to use this program as a source of code for a general
# API for selecting particular data concerning any country in the downloadable World Bank WDI
# data and producing a study table in an SQLite database for the topic which the selection of WDI
# is relevant to. Each country should have its own table in such a database for clarity of the
# SQL used to produce reports and the Python or R used to generate viauals.
#
import math
from collections import OrderedDict
from os import environ
from sqlite3 import connect
from sys import stderr
from os.path import isdir, isfile
from datetime import datetime

import pandas

MODULE_NAME    = "MIS_505_Portfolio_Project"
TESTING       = True
DEBUG         = False

class CountryData:

    CountryIndicatorDbFileName = "CountryIndicatorDb"
    DataFolder = environ['HOME'] + '/ACTIVITIES/Graduate School/CSU Global/Advanced Data Analytics
Certificate/Courses/MIS 505 - Data Wrangling/Assignments/Week 8/production data'
    WDIDataCSVFileName = '/WDIData.csv'

    @staticmethod
    def initializeStorage():
        dbFilePath = CountryData.DataFolder + '/' + CountryData.CountryIndicatorDbFileName
        connection = connect(dbFilePath)
        connection.execute("""CREATE TABLE IF NOT EXISTS "IndicatorNameMap" (
```

```

        "Indicator Code"          TEXT NOT NULL,
        "Indicator Name"         TEXT NOT NULL)""")
if PortfolioProject.WDI_DATA_FRAME is None:
    wdiDataFrame = PortfolioProject.load_WDI()
else:
    wdiDataFrame = PortfolioProject.WDI_DATA_FRAME
if wdiDataFrame is None:
    return False
# Produce parallel data lists from columns: 'Country Name', 'Indicator Name',
# 'Indicator Code'
countryNames = wdiDataFrame['Country Name'].to_list()
rowCountCountryNames = len(countryNames)
indicatorName = wdiDataFrame['Indicator Name'].to_list()
rowCountIndicatorName = len(indicatorName)
indicatorCode = wdiDataFrame['Indicator Code'].to_list()
rowCountIndicatorCode = len(indicatorCode)
if rowCountCountryNames != rowCountIndicatorName:
    print("\nWarning: The row count for the Country Names column is not equal to the row count for the
Indicator Name column", file=stderr)
if rowCountCountryNames != rowCountIndicatorCode:
    print("\nWarning: The row count for the Country Name column is not equal to the row count for the
Indicator Code", file=stderr)
if rowCountIndicatorName != rowCountIndicatorCode:
    print("\nWarning: The row count for the Indicator Name column is not equal to the row count for the
Indicator Code column", file=stderr)
# No warnings were printed on 2026-09-04
# Still assuming they are parallel regardless of warnings, use minimum length for
# row count:
rowCount = min(rowCountCountryNames, rowCountIndicatorName, rowCountIndicatorCode)

# Data Quality Check:
# Check to see if all of the indicator codes present in any one country are
# present in all.
# This ran with zero warnings on 2026-09-04.
countryNameToWDIlistMap = {}
codeCount = 1442
rowIdx = 0
currentCountry = None
sampleCodeList = None
while rowIdx < rowCount:
    if countryNames[rowIdx] not in countryNameToWDIlistMap:
        if currentCountry is not None:
            if sampleCodeList is None:
                sampleCodeList = countryNameToWDIlistMap[countryNames[rowIdx-1]]
            else:
                if sampleCodeList != countryNameToWDIlistMap[countryNames[rowIdx-1]]:
                    print("Warning: Indicator Code lists for two countries are not the same",
file=stderr)
        else:
            currentCountry = countryNames[rowIdx]
            countryNameToWDIlistMap[countryNames[rowIdx]] = []
    if sampleCodeList is None:
        sampleCodeList
    countryNameToWDIlistMap[countryNames[rowIdx]].append(indicatorCode[rowIdx])
    rowIdx += 1
# Construct table mapping all of the WDI codes to their names for use in reports built
# using the codes as column names or row selectors, depending on the database format.
indicatorCode = indicatorCode[:codeCount]
indicatorName = indicatorName[:codeCount]
rowIdx = 0
while rowIdx < codeCount:
    connection.execute("""INSERT INTO "IndicatorNameMap"
        ("Indicator Code", "Indicator Name")
        VALUES ("{indicatorCode}", "{indicatorName}")""").
        format(indicatorCode=indicatorCode[rowIdx],
            indicatorName=indicatorName[rowIdx]))
    rowIdx += 1
connection.commit()
connection.close()
return True

@staticmethod
def getIndicatorNameMap():
    """
    Load the IndicatorNameMap table and build maps which provide the WDI Code for any WDI Name and
    the WDI Name for any WDI Code.
    :return: nameToCodeMap, codeToNameMap
    """

```

```

dbFilePath = CountryData.DataFolder + '/' + CountryData.CountryIndicatorDbFileName
connection = connect(dbFilePath)
entries = connection.execute("""SELECT * FROM IndicatorNameMap;""").fetchall()
connection.close()

nameToCodeMap = OrderedDict()
codeToNameMap = OrderedDict()
for entry in entries:
    nameToCodeMap[entry[1]] = entry[0]
    codeToNameMap[entry[0]] = entry[1]
return nameToCodeMap, codeToNameMap

class PortfolioProject:

    DataFolder = environ['HOME'] + '/ACTIVITIES/Graduate School/CSU Global/Advanced Data Analytics
Certificate/Courses/MIS 505 - Data Wrangling/Assignments/Week 8/data downloaded'
    WDI_CSVFileName = '/WDIData.csv'
    PRM_ENRL_CSVFileName = '/API_SE.PRM.ENRL_DS2_en_csv_v2_4538564.csv'

    WDI_DATA_FRAME = None
    WDI_SOURCE_DATA = None
    PRM_ENRL_DATA_FRAME = None

    @staticmethod
    def load_PRM_ENRL():
        prmEnrlDataFrame = None
        enrollmentDataFilePath = PortfolioProject.DataFolder + PortfolioProject.PRM_ENRL_CSVFileName
        if not isfile(enrollmentDataFilePath):
            print("\nDownloaded data file does not exist:\t" + enrollmentDataFilePath, file=stderr)
        else:
            try:
                prmEnrlDataFrame = pandas.read_csv(enrollmentDataFilePath, skiprows=4, header=0)
                columnNames = prmEnrlDataFrame.columns.values.tolist()
                print("\nNames of columns in the PRM ENRL DataFrame:", columnNames)
                print('\nPrmEnrlDataFrame:\n', prmEnrlDataFrame.head(10))
            except Exception as ex:
                print("\nPandas construction of EDU DataFrame from CSV file failed with:")
                print(ex)
        PortfolioProject.PRM_ENRL_DATA_FRAME = prmEnrlDataFrame
        return prmEnrlDataFrame

    @staticmethod
    def pivotEduData(countryName: str):
        if PortfolioProject.PRM_ENRL_DATA_FRAME is None:
            eduCsvFilePath = PortfolioProject.DataFolder + PortfolioProject.PRM_ENRL_CSVFileName
            eduDataFrame = pandas.read_csv(eduCsvFilePath, skiprows=4)
        else:
            eduDataFrame = PortfolioProject.PRM_ENRL_DATA_FRAME
        # Extract the data for the particular country
        countryDF = eduDataFrame[eduDataFrame['Country Name'] == countryName]
        if DEBUG:
            print("\ncountryDF:\n", countryDF.head(10))
        columnNames = eduDataFrame.columns.to_list()
        yearList = []
        wdiList = []
        for columnName in columnNames:
            if columnName.isnumeric():
                yearList.append(int(columnName))
                wdiValue = countryDF[columnName].values[0]
                if not math.isnan(wdiValue):
                    wdiList.append(countryDF[columnName].values[0])
            else:
                wdiList.append(None)
        return countryDF, yearList, wdiList

    @staticmethod
    def load_WDI_Data(refresh: bool=False):
        """
        Load the data for the particular named country corresponding to the listed wdiCodes.
        :param countryName:
        :param wdiCode:
        :return:
        """
        if not PortfolioProject.WDI_SOURCE_DATA is None and not refresh:
            return PortfolioProject.WDI_SOURCE_DATA
        wdiDataFrame = None
        WDIDataCSVFilePath = PortfolioProject.DataFolder + PortfolioProject.WDI_CSVFileName

```

```

if not isfile(WDIDataCSVFilePath):
    print("\nDownloaded data file does not exist:\t" + WDIDataCSVFilePath, file=stderr)
else:
    try:
        startTime = datetime.now()
        wdiDataFrame = pandas.read_csv(WDIDataCSVFilePath)
        print("\nLoaded WDI CSV file into Pandas DataFrame:")
        print(wdiDataFrame.head(10))
        print("\nUnique Country Names in WDI DataFrame:", wdiDataFrame['Country
Name'].unique().tolist())
    except Exception as ex:
        print("\nPandas construction of WDI DataFrame from CSV file failed with:")
        print(ex)
    finally:
        endTime = datetime.now()
        loadTime = endTime - startTime
        print("\nTime required to load 209 MB WDI CSV data file:\t", loadTime)
    columnNames = wdiDataFrame.columns.tolist()
    PortfolioProject.WDI_DATA_FRAME = wdiDataFrame

    PortfolioProject.WDI_SOURCE_DATA = {
        'columnNames': columnNames,
        'wdiDataFrame': wdiDataFrame
    }
    return PortfolioProject.WDI_SOURCE_DATA

@staticmethod
def makeAssignment8_DB_n_CSV(countryName: str= "Argentina", force: bool=False):
    """
    Make a separate DB file named Assignment8_[country].sqllitedb
    with a single table in it named Assignment8_[country]
    and a separate CSV file named Assignment8_[country].csv.
    :param countryName:
    :param force:
    :return:
    """
    countryDbPath = CountryData.DataFolder + '/' + 'Assignment8_' + countryName + '.sqllitedb'
    tableName = 'Assignment8_' + countryName
    report = {}

    # If table already exists, do not rewrite unless force==True
    connection = connect(countryDbPath)
    tableNamePresent = connection.execute("""SELECT name from sqlite_master WHERE
type='table';""").fetchall()
    if len(tableNamePresent) > 0 and tableNamePresent[0][0] == tableName:
        if not force:
            report['Error'] = "Table for specified country already exists:\t" + tableName
            return report
        else:
            connection.execute("""DROP TABLE "{}";""".format(tableName))

    connection.execute("""CREATE TABLE IF NOT EXISTS "{}"
        ("Year" INTEGER NOT NULL,
        "Primary Enrollment" NUMERIC,
        "GDP" NUMERIC);""".format(tableName=tableName))

    connection.close()

    countryDF, yearList, wdiList = PortfolioProject.pivotEduData(countryName)

    # Using 'GDP (current US$)' from WDI Data: 'NY.GDP.MKTP.CD' is the 'Indicator Code'.
    columnNames = PortfolioProject.WDI_SOURCE_DATA['columnNames']
    wdiDataFrame = PortfolioProject.WDI_SOURCE_DATA['wdiDataFrame']
    if DEBUG:
        print("\nwdiDataFrame:\n", wdiDataFrame)

    dataFrame = wdiDataFrame[(wdiDataFrame[columnNames[0]] == countryName) & (wdiDataFrame['Indicator
Code'] == 'NY.GDP.MKTP.CD')]
    values = list(dataFrame.values)
    tableData = []
    rowIdx = 0
    while rowIdx < len(values[0]):
        if columnNames[rowIdx].isnumeric():
            try:
                val = float(values[0][rowIdx])
            except:
                val = None
            tableData.append((int(columnNames[rowIdx]), val))
        rowIdx += 1

```

```

if DEBUG:
    print('\nExtracted GDP Data:\n', dataframe.head(25))
    print('\tDimensions:\t', dataframe.size)
    print('\t')
    # for value in dataframe.values:
    #     print(value)

if DEBUG:
    rowIdx = 0
    while rowIdx < len(tableData):
        print(str(rowIdx) + '\tyearList[rowIdx]:\t' + str(yearList[rowIdx]) +
              '\twdiList[rowIdx]:\t' + str(wdiList[rowIdx]) + '\ttableData[rowIdx]:\t' +
str(tableData[rowIdx]))
        rowIdx += 1
        # yearList, wdiList, tableData

connection = connect(countryDbPath)
rowIdx = 0
while rowIdx < len(tableData):
    connection.execute("""INSERT INTO "{tableName}" ("Year", "Primary Enrollment", "GDP")
                        VALUES (?, ?, ?);""".format(tableName=tableName,
                                                    (yearList[rowIdx], wdiList[rowIdx], tableData[rowIdx][1]))

    rowIdx += 1
connection.commit()
connection.close()

# Produce the CSV file from the table:
countryCsvPath = CountryData.DataFolder + '/' + 'Assignment8_' + countryName + '.csv'
if isfile(countryCsvPath):
    if not force:
        report['Error'] = "Table for specified country already exists:\t" + tableName
        return report
connection = connect(countryDbPath)
dataFrame = pandas.read_sql("""SELECT * FROM "{tableName}";""".format(tableName, connection)
dataFrame.to_csv(countryCsvPath, index=False)

if __name__ == "__main__":
    print("\nRunning:\t" + MODULE_NAME)

    if not isdir(PortfolioProject.DataFolder):
        print("\nFolder containing downloaded data does not exist:\t" + PortfolioProject.DataFolder,
file=stderr)

    PortfolioProject.load_WDI_Data()
    PortfolioProject.load_PRN_ENRL()
    CountryData.initializeStorage()
    PortfolioProject.makeAssignment_8_DB_n_CSV('Argentina')

```

[END OF CODE]

Running: MIS_505_Portfolio_Project

Loaded WDI CSV file into Pandas DataFrame:

	Country Name	Country Code	\
0	Africa Eastern and Southern	AFE	
1	Africa Eastern and Southern	AFE	
2	Africa Eastern and Southern	AFE	
3	Africa Eastern and Southern	AFE	
4	Africa Eastern and Southern	AFE	
5	Africa Eastern and Southern	AFE	
6	Africa Eastern and Southern	AFE	
7	Africa Eastern and Southern	AFE	
8	Africa Eastern and Southern	AFE	
9	Africa Eastern and Southern	AFE	

Indicator Name Indicator Code 1960

\			
0	Access to clean fuels and technologies for coo...	EG.CFT.ACCS.ZS	NaN
1	Access to clean fuels and technologies for coo...	EG.CFT.ACCS.RU.ZS	NaN
2	Access to clean fuels and technologies for coo...	EG.CFT.ACCS.UR.ZS	NaN
3	Access to electricity (% of population)	EG.ELC.ACCS.ZS	NaN
4	Access to electricity, rural (% of rural popul...	EG.ELC.ACCS.RU.ZS	NaN
5	Access to electricity, urban (% of urban popul...	EG.ELC.ACCS.UR.ZS	NaN
6	Account ownership at a financial institution o...	FX.OWN.TOTL.ZS	NaN
7	Account ownership at a financial institution o...	FX.OWN.TOTL.FE.ZS	NaN
8	Account ownership at a financial institution o...	FX.OWN.TOTL.MA.ZS	NaN
9	Account ownership at a financial institution o...	FX.OWN.TOTL.OL.ZS	NaN

	1961	1962	1963	1964	1965	...	2013	2014	2015	\
0	NaN	NaN	NaN	NaN	NaN	...	16.936004	17.337896	17.687093	
1	NaN	NaN	NaN	NaN	NaN	...	6.499471	6.680066	6.859110	
2	NaN	NaN	NaN	NaN	NaN	...	37.855399	38.046781	38.326255	
3	NaN	NaN	NaN	NaN	NaN	...	31.794160	32.001027	33.871910	
4	NaN	NaN	NaN	NaN	NaN	...	18.663502	17.633986	16.464681	
5	NaN	NaN	NaN	NaN	NaN	...	67.112206	66.283426	67.080235	
6	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
7	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
8	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
9	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	

	2016	2017	2018	2019	2020	2021	Unnamed: 66
0	18.140971	18.491344	18.825520	19.272212	19.628009	NaN	NaN
1	7.016238	7.180364	7.322294	7.517191	7.651598	NaN	NaN
2	38.468426	38.670044	38.722783	38.927016	39.042839	NaN	NaN
3	38.880173	40.261358	43.061877	44.270860	45.803485	NaN	NaN
4	24.531436	25.345111	27.449908	29.641760	30.404935	NaN	NaN
5	69.132292	70.928567	71.866136	73.332842	73.942949	NaN	NaN
6	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	NaN	NaN	NaN	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN	NaN	NaN	NaN

[10 rows x 67 columns]

Unique Country Names in WDI DataFrame: ['Africa Eastern and Southern', 'Africa Western and Central', 'Arab World', 'Caribbean small states', 'Central Europe and the Baltics', 'Early-demographic dividend', 'East Asia & Pacific', 'East Asia & Pacific (excluding high income)', 'East Asia & Pacific (IDA & IBRD countries)', 'Euro area', 'Europe & Central Asia', 'Europe & Central Asia (excluding high income)', 'Europe & Central Asia (IDA & IBRD countries)', 'European Union', 'Fragile and conflict affected situations', 'Heavily indebted poor countries (HIPC)', 'High income', 'IBRD only', 'IDA & IBRD total', 'IDA blend', 'IDA only', 'IDA total', 'Late-demographic dividend', 'Latin America & Caribbean', 'Latin America & Caribbean (excluding high income)', 'Latin America & the Caribbean (IDA & IBRD countries)', 'Least developed countries: UN classification', 'Low & middle income', 'Low income', 'Lower middle income', 'Middle East & North Africa', 'Middle East & North Africa (excluding high income)', 'Middle East & North Africa (IDA & IBRD countries)', 'Middle income', 'North America', 'Not classified', 'OECD members', 'Other small states', 'Pacific island small states', 'Post-

demographic dividend', 'Pre-demographic dividend', 'Small states', 'South Asia', 'South Asia (IDA & IBRD)', 'Sub-Saharan Africa', 'Sub-Saharan Africa (excluding high income)', 'Sub-Saharan Africa (IDA & IBRD countries)', 'Upper middle income', 'World', 'Afghanistan', 'Albania', 'Algeria', 'American Samoa', 'Andorra', 'Angola', 'Antigua and Barbuda', 'Argentina', 'Armenia', 'Aruba', 'Australia', 'Austria', 'Azerbaijan', 'Bahamas, The', 'Bahrain', 'Bangladesh', 'Barbados', 'Belarus', 'Belgium', 'Belize', 'Benin', 'Bermuda', 'Bhutan', 'Bolivia', 'Bosnia and Herzegovina', 'Botswana', 'Brazil', 'British Virgin Islands', 'Brunei Darussalam', 'Bulgaria', 'Burkina Faso', 'Burundi', 'Cabo Verde', 'Cambodia', 'Cameroon', 'Canada', 'Cayman Islands', 'Central African Republic', 'Chad', 'Channel Islands', 'Chile', 'China', 'Colombia', 'Comoros', 'Congo, Dem. Rep.', 'Congo, Rep.', 'Costa Rica', 'Cote d'Ivoire', 'Croatia', 'Cuba', 'Curacao', 'Cyprus', 'Czech Republic', 'Denmark', 'Djibouti', 'Dominica', 'Dominican Republic', 'Ecuador', 'Egypt, Arab Rep.', 'El Salvador', 'Equatorial Guinea', 'Eritrea', 'Estonia', 'Eswatini', 'Ethiopia', 'Faroe Islands', 'Fiji', 'Finland', 'France', 'French Polynesia', 'Gabon', 'Gambia, The', 'Georgia', 'Germany', 'Ghana', 'Gibraltar', 'Greece', 'Greenland', 'Grenada', 'Guam', 'Guatemala', 'Guinea', 'Guinea-Bissau', 'Guyana', 'Haiti', 'Honduras', 'Hong Kong SAR, China', 'Hungary', 'Iceland', 'India', 'Indonesia', 'Iran, Islamic Rep.', 'Iraq', 'Ireland', 'Isle of Man', 'Israel', 'Italy', 'Jamaica', 'Japan', 'Jordan', 'Kazakhstan', 'Kenya', 'Kiribati', 'Korea, Dem. People's Rep.', 'Korea, Rep.', 'Kosovo', 'Kuwait', 'Kyrgyz Republic', 'Lao PDR', 'Latvia', 'Lebanon', 'Lesotho', 'Liberia', 'Libya', 'Liechtenstein', 'Lithuania', 'Luxembourg', 'Macao SAR, China', 'Madagascar', 'Malawi', 'Malaysia', 'Maldives', 'Mali', 'Malta', 'Marshall Islands', 'Mauritania', 'Mauritius', 'Mexico', 'Micronesia, Fed. Sts.', 'Moldova', 'Monaco', 'Mongolia', 'Montenegro', 'Morocco', 'Mozambique', 'Myanmar', 'Namibia', 'Nauru', 'Nepal', 'Netherlands', 'New Caledonia', 'New Zealand', 'Nicaragua', 'Niger', 'Nigeria', 'North Macedonia', 'Northern Mariana Islands', 'Norway', 'Oman', 'Pakistan', 'Palau', 'Panama', 'Papua New Guinea', 'Paraguay', 'Peru', 'Philippines', 'Poland', 'Portugal', 'Puerto Rico', 'Qatar', 'Romania', 'Russian Federation', 'Rwanda', 'Samoa', 'San Marino', 'Sao Tome and Principe', 'Saudi Arabia', 'Senegal', 'Serbia', 'Seychelles', 'Sierra Leone', 'Singapore', 'Sint Maarten (Dutch part)', 'Slovak Republic', 'Slovenia', 'Solomon Islands', 'Somalia', 'South Africa', 'South Sudan', 'Spain', 'Sri Lanka', 'St. Kitts and Nevis', 'St. Lucia', 'St. Martin (French part)', 'St. Vincent and the Grenadines', 'Sudan', 'Suriname', 'Sweden', 'Switzerland', 'Syrian Arab Republic', 'Tajikistan', 'Tanzania', 'Thailand', 'Timor-Leste', 'Togo', 'Tonga', 'Trinidad and Tobago', 'Tunisia', 'Turkiye', 'Turkmenistan', 'Turks and Caicos Islands', 'Tuvalu', 'Uganda', 'Ukraine', 'United Arab Emirates', 'United Kingdom', 'United States', 'Uruguay', 'Uzbekistan', 'Vanuatu', 'Venezuela, RB', 'Vietnam', 'Virgin Islands (U.S.)', 'West Bank and Gaza', 'Yemen, Rep.', 'Zambia', 'Zimbabwe']

Time required to load 209 MB WDI CSV data file: 0:00:03.219826

Names of columns in the PRM ENRL DataFrame: ['Country Name', 'Country Code', 'Indicator Name', 'Indicator Code', '1960', '1961', '1962', '1963', '1964', '1965', '1966', '1967', '1968', '1969', '1970', '1971', '1972', '1973', '1974', '1975', '1976', '1977', '1978', '1979', '1980', '1981', '1982', '1983', '1984', '1985', '1986', '1987', '1988', '1989', '1990', '1991', '1992', '1993', '1994', '1995', '1996', '1997', '1998', '1999', '2000', '2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008', '2009', '2010', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018']

'2019', '2020', '2021', 'Unnamed: 66']

PrmEnrldataFrame:

	Country Name	Country Code	Indicator Name	\
0	Aruba	ABW	Primary education, pupils	
1	Africa Eastern and Southern	AFE	Primary education, pupils	
2	Afghanistan	AFG	Primary education, pupils	
3	Africa Western and Central	AFW	Primary education, pupils	
4	Angola	AGO	Primary education, pupils	
5	Albania	ALB	Primary education, pupils	
6	Andorra	AND	Primary education, pupils	
7	Arab World	ARB	Primary education, pupils	
8	United Arab Emirates	ARE	Primary education, pupils	
9	Argentina	ARG	Primary education, pupils	

	Indicator Code	1960	1961	1962	1963	1964	1965	...	2013	\
0	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	9803.0	
1	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
2	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	5986268.0	
3	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
4	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
5	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	198897.0	
6	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	4068.0	
7	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	43561852.0	
8	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	376649.0	
9	SE.PRM.ENRL	NaN	NaN	NaN	NaN	NaN	NaN	...	4791544.0	

	2014	2015	2016	2017	2018	2019	\
0	9816.0	NaN	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	NaN	NaN	
2	6217756.0	6199329.0	6265011.0	6350404.0	6544906.0	6777785.0	
3	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	5620915.0	NaN	NaN	NaN	NaN	
5	195720.0	188371.0	179564.0	174836.0	170861.0	167104.0	
6	4065.0	4164.0	4248.0	4263.0	4325.0	4322.0	
7	44953900.0	45545024.0	46378904.0	47415468.0	48693064.0	50097480.0	
8	409776.0	460975.0	467723.0	486086.0	NaN	553678.0	
9	4780105.0	4784446.0	4775782.0	4753843.0	4776452.0	4800419.0	

	2020	2021	Unnamed: 66
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	NaN	NaN	NaN
5	162170.0	NaN	NaN
6	4246.0	NaN	NaN
7	51525644.0	NaN	NaN
8	462333.0	NaN	NaN
9	NaN	NaN	NaN

[10 rows x 67 columns]

